

DMA

DMA module is used to initiate the connectivity of wishbone interface and ensures the transfer of data between them. DMA has a modport in top-level interface. This module interfaces with channel selector, regfile and also wishbone interface. The trigger to DMA comes from channel selector and this establishes connection between wishbone modules reading the register contents. Counter module is instantiated in this DMA. Generate statements are used again to instantiate them.

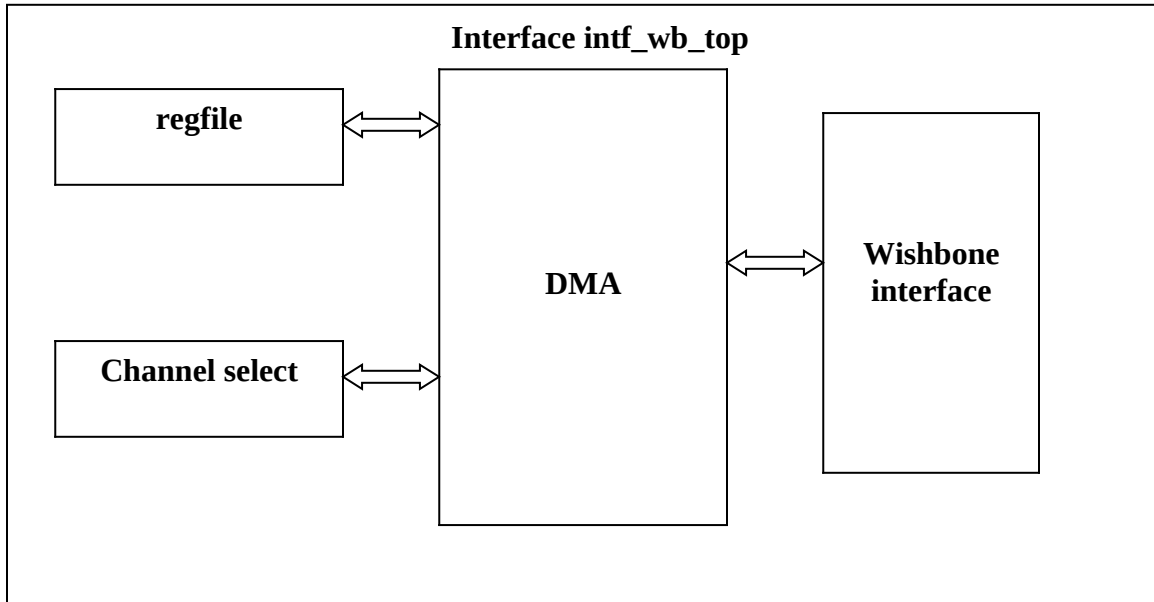


Fig 1. Interface connectivity

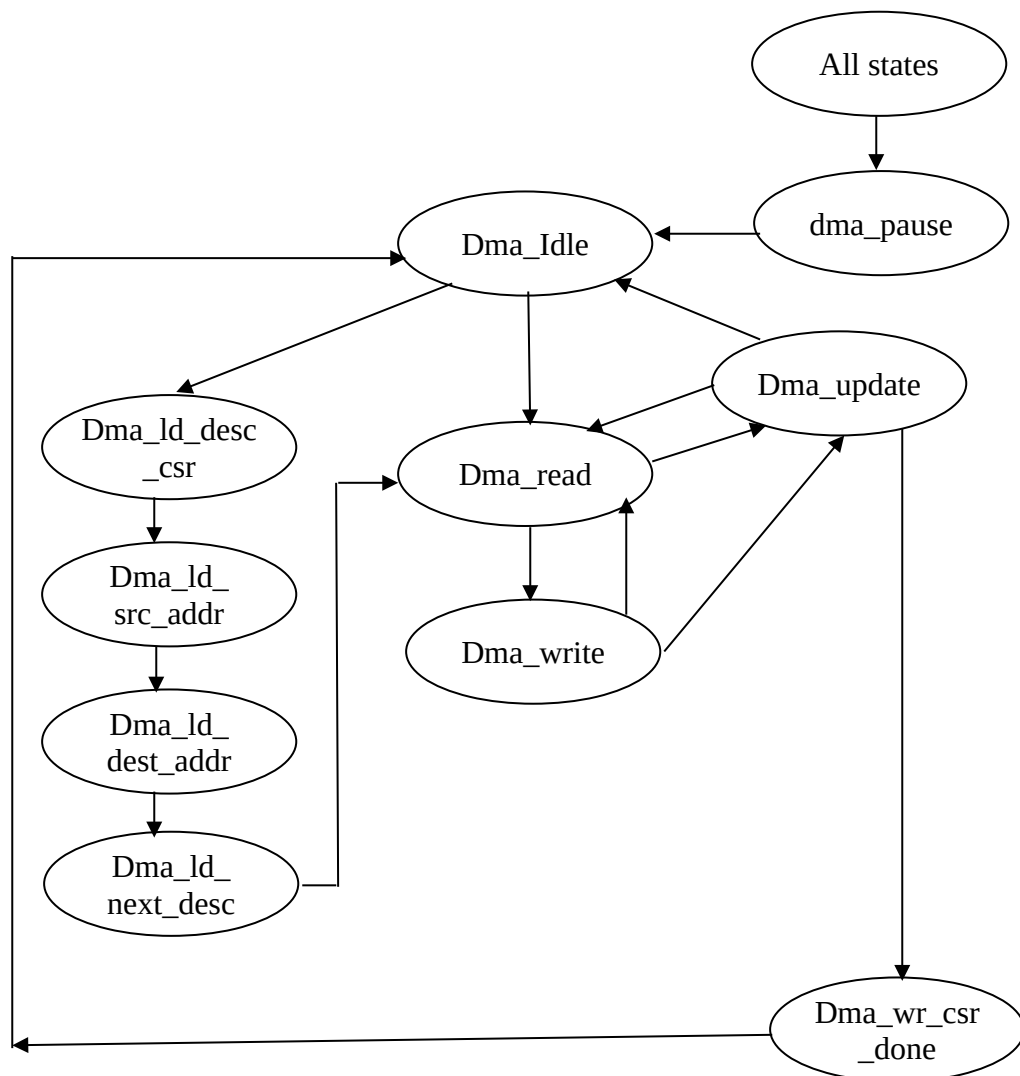


Fig 2 State machine for DMA engine

State machine

dma_Idle : On reset this state is reached. All the variables are assigned their initial values in this state.

dma_idle -> dma_pause : When pause request is asserted all the states should move to dma_pause state.

dma_idle -> dma_read : when dma_start and ch_csr_reg[USE_ED] bit is zero then, the state transition happens.

dma_idle -> dma_ld_desc_csr : When dma_start and ch_csr_reg[USE_ED] bit is one then, the state transition occurs.

dma_pause : All the states should have access to this state. When csr_reg[PAUSE] is asserted all the states should come to this state.

dma_pause -> dma_idle : When csr_reg[PAUSE] is de-asserted, the state machine should start and go back to idle.

dma_update : During read or write if ch_csr_reg[STOP] bit is set, then the states transit to this. If the transfer is complete then also all states come to this to update registers. In this state, the error is asserted and goes back to idle state if ch_csr_reg[STOP] bit is set. If stop is not set, the state will update the required registers.

In this state, the check has to be done to see if ch_csr_reg[ARS] bit is set and if so, then the ch_csr_reg[REST] is also set then, the state should go to read state.

dma_update -> dma_idle : Assert ch_csr_reg[ERR] bit and go back to idle.

dma_update -> dma_wr_csr_done : When desc_reg[EOL] bit is set, the control should flow like this.

dma_update -> dma_read : when ch_csr_reg[ARS] == 1 && ch_csr_reg[REST] == 1 then, this should happen to restart the dma transfer.

dma_read : For read access this state is reached.

dma_read -> dma_write : When master_drdy is asserted, the transfer goes to write.

dma_read -> dma_update : when ch_csr_reg[STOP] bit is set, the DMA transfer is aborted and error signal is asserted.

dma_write : For write access this state is reached.

dma_write -> dma_update : When ch_csr_reg[STOP] bit is set or when chunk transfer is complete.

dma_write -> dma_read : When chunk of transfer is not happened, then it goes back to read state.

dma_ld_desc_csr : When ch_csr_reg[USE_ED] is set, then from idle state the transfer happens. ch_csr_reg[4:1] = descriptor_csr[19:16]

ch_csr_reg[11:0] = descriptor_csr[11:0]

dma_ld_desc_csr -> dma_ld_src_addr : When master_drdy is set.

dma_ld_src_addr : writing of source address will happen here.

ch_csr_src_addr = descriptor_src_addr

dma_ld_src_addr -> dma_ld_dest_addr : When master_drdy is set.

dma_ld_dest_addr : writing of destination address will happen here.

ch_csr_dest_addr = descriptor_dest_addr

dma_ld_dest_addr -> dma_ld_next_addr : When master_drdy is set.

dma_ld_next_addr : writing of next descriptor address will happen here.

ch_desc = descriptor_next_addr

dma_ld_next_addr -> dma_ld_desc_csr : When master_drdy is set.

dma_ls_next_addr -> dma_read : If descriptor_csr[EOL] bit is set then, the state moves here.

dma_wr_csr_done : writes the ch_csr_reg[DONE] bit and goes back to idle.

dma_wr_csr_done -> dma_idle : next cycle.

Table 5 in appendix, shows the control signal flow in dma engine

The ch_csr_src_sel and ch_csr_dest_sel bits indicate which master and which slave are accessed.

Address incrementing in the dma engine is done using circular buffers fashion. Circular buffer is the one in which the buffers never go beyond allocated memory space. These buffers will wrap around when last entry in the buffer is reached. Size of address mask register is 28 bits. The upper 28 bits in address location are incremented only if the mask bit for that particular bit is set.

Address increment is done using another counter module in which the 15 bits data is incremented in first phase then, the rest 15 bits are added to it to improve the performance of a counter.

Once the Address is incremented the next address[31:4] is the value of the incremented address[31:4] only if mask[31:4] bits are asserted. Each bit is compared separately.

If the ch_sft_ptr[swptr_en] is set then, the DMA engine has to always compare the software pointer value with the incremented address. If both are equal or if incremented value is more than the one in software pointer register, then, the write operation will halt until software pointer is updated. Software should take care of this updating.

The DMA done signal will be asserted once the chunk of data is transmitted. If total size of the transfer is over then, dma all done signal will be asserted. For all this assertion a counter is required to count the data that's been transferred. After every valid transfer the counter should be incremented to check the number of words transferred. If the transfer reaches the chunk size then, the dma done signal is asserted. If the transfer reaches the total size then, the dma all done signal is asserted.

ch_csr_reg[REST] bit is enabled along with ch_csr_reg[ARS], then in the update state the check has to be done and the flow should go to read state until the bit is cleared.

The port connections are described in the table below. The dma is a modport in the top-level interface so, all the signals in port lists are members of modport in top. Top is instantiated as u_dma_eng.

Signal name	Width	Description
<i>Inputs</i>		
i_clk	1	Global Clock
i_reset	1	Global Reset
i_dma_start	1	From channel select indicating the DMA to start
i_dma_nd	1	Next descriptor
i_req	1	Input from Host indicating to start the DMA hardware handshake mode
i_ch_csr	32	Channel CSR value from regfile
i_ch_sz	32	Channel transfer size from regfile
i_ch_addr0	32	Channel address0 from regfile
i_ch_am0	32	Channel address0 mask from regfile
i_ch_addr1	32	Channel address1 from regfile
i_ch_am1	32	Channel address1 mask from regfile
i_ch_desc	32	Channel linked list descriptor pointer from regfile
i_ch_swptr	32	Channel software pointer from regfile
i_dma_stop	1	From regfile ch_csr_reg[STOP]
i_dma_pause	1	From regfile csr_reg[PAUSE]
i_mast0_data	32	Master0 data
i_mast0_err	1	Master error bit
i_mast0_drdy	1	Master0 data ready signal
i_mast1_data	32	Master1 data
i_mast1_err	1	Master1 error bit
i_mast1_drdy	1	Master1 data ready signal
<i>Outputs</i>		
o_ch_csr	32	Channel CSR value from regfile
o_ch_addr0	32	Channel address0 to regfile
o_ch_addr1	32	Channel address1 to regfile
o_ch_next_desc	32	Channel linked list descriptor next pointer to regfile
o_ch_csr_we	1	Channel CSR value to regfile
o_ch_addr0_we	1	Channel address0 to regfile
o_ch_addr1_we	1	Channel address1 to regfile
o_ch_next_desc_we	1	Channel linked list descriptor pointer to regfile
o_status_fetch_desc	1	Status indicating the fetching descriptor
o_mast0_start	1	Start master0
o_mast0_wait	1	Insert wait states
o_mast0_wr_n_rd	1	If set write else read
o_mast0_data	32	Master data out
o_mast0_addr	32	Master address
o_mast1_start	1	Start master1
o_mast1_wait	1	Insert wait states

Signal name	Width	Description
o_mast1_wr_n_rd	1	If set write else read
o_mast1_data	32	Master data out
o_mast1_addr	32	Master address
o_dma_ack	1	Acknowledge to dma
o_dma_nxt_desc	1	Next descriptor
o_dma_bsy	1	Status dma busy
o_dma_done	1	Dma chunk transfer done
o_dma_done_all	1	Complete Dma transfer is over
o_dma_err	1	Error status when ch_csr_reg[STOP] is asserted

Table 5 showing the DMA input output signals